

INF 662 · DESARROLLO DE APLICACIONES MÓVILES

Guía de Laboratorio N.º 8

Ejemplo integrador: cronómetro con historial

MATERIA INF 662 — Desarrollo de Aplicaciones Móviles**SEMESTRE** 6.º · Mención Ing. de Software**TEMA** Ejemplo resuelto de consolidación**DURACIÓN** 1 sesión de laboratorio**CAPÍTULOS** Consolida Cap. 1–3 (Guías 1 a 5)**MODALIDAD** Laboratorio · individual**DOCENTE** M. Sc. Lic. Huáscar Fedor Gonzales Guzmán**REQUISITO** Guías de Laboratorio N.º 1 a N.º 5**NOTA · Tercer ejemplo integrador**

Este ejemplo reúne lo aprendido y suma una pieza nueva pero sencilla: el manejo del **tiempo** con **Timer**. Construirás un **cronómetro** con iniciar, pausar y reiniciar, que además guarda vueltas (laps) en una lista con historial.

01 Objetivos

Objetivo general. Construir un cronómetro que actualiza su valor en el tiempo, integrando el manejo de estado, un temporizador periódico y una lista dinámica de vueltas.

Objetivos específicos:

- Usar `Timer.periodic` para actualizar la interfaz a intervalos regulares.
- Controlar el ciclo de vida del temporizador (crear, cancelar) y liberarlo en `dispose`.
- Manejar estado con `setState` para el tiempo, el modo (corriendo/pausado) y las vueltas.
- Formatear el tiempo en minutos:segundos:centésimas.
- Registrar y mostrar las vueltas con `ListView.builder`.

02 Análisis del problema

El cronómetro cuenta el tiempo transcurrido en centésimas de segundo. Un `Timer.periodic` que dispara cada 100 ms suma tiempo y refresca la pantalla. Los botones cambian el estado:

EJEMPLO · Estados y acciones

Iniciar / Pausar: arranca o detiene el temporizador (un solo botón que alterna).

Vuelta (lap): guarda el tiempo actual en la lista de vueltas (solo si está corriendo).

Reiniciar: cancela el temporizador, pone el tiempo en cero y vacía las vueltas.

DEFINICIÓN · Timer.periodic

`Timer.periodic(duracion, callback)` ejecuta el `callback` repetidamente cada `duracion`. Devuelve un objeto `Timer` que se debe guardar para poder cancelarlo con `.cancel()`.

03 Requisitos previos

Requisito indispensable. Haber completado las Guías N.º 1 a N.º 5.

Conceptos que se reutilizan y se amplían:

StatefulWidget + setState

dispose

ListView.builder

Timer (dart:async)

Row de botones

Formateo de texto

ADVERTENCIA · Importa dart:async

La clase `Timer` vive en la biblioteca `dart:async`. Debes agregar `import 'dart:async';` al inicio del archivo, junto al import de Material.

04 Desarrollo · Parte A — Estado y temporizador

1 Preparar la pantalla con estado

1. Crea el proyecto `cronometro_app`. En `lib/main.dart`, recuerda los dos imports y crea el `StatefulWidget` con su estado:

```
LIB/MAIN.DART · DART (IMPORTS)
import 'dart:async';
import 'package:flutter/material.dart';
```

```
LIB/MAIN.DART · DART
class Cronometro extends StatefulWidget {
  const Cronometro({super.key});

  @override
  State<Cronometro> createState() => _CronometroState();
}

class _CronometroState extends State<Cronometro> {
  Timer? _timer;
  int _centesimas = 0;
  bool _corriendo = false;
  final List<String> _vueltas = [];

  @override
  void dispose() {
    _timer?.cancel();
    super.dispose();
  }
}
```

```
}
```

ADVERTENCIA · Cancelar el Timer en dispose

Si la pantalla se cierra con el cronómetro activo, el `Timer` seguiría ejecutándose e intentaría llamar a `setState` sobre un widget destruido. Cancelarlo en `dispose` evita ese error.

2 Iniciar y pausar

1. Un solo método alterna entre correr y pausar. Al correr, crea el `Timer.periodic`; al pausar, lo cancela:

LIB/MAIN.DART · DART

```
void _iniciarPausar() {
  if (_corriendo) {
    _timer?.cancel();
    setState(() => _corriendo = false);
  } else {
    _timer = Timer.periodic(
      const Duration(milliseconds: 100),
      (timer) => setState(() => _centesimas++),
    );
    setState(() => _corriendo = true);
  }
}
```

3 Vuelta y reiniciar

1. Agrega los métodos para guardar una vuelta y para reiniciar todo:

LIB/MAIN.DART · DART

```
void _vuelta() {
  if (!_corriendo) return;
  setState(() => _vueltas.insert(0, _formatear(_centesimas)));
}

void _reiniciar() {
  _timer?.cancel();
  setState(() {
    _centesimas = 0;
    _corriendo = false;
    _vueltas.clear();
  });
}
```

NOTA · insert(0, ...) para el historial

`_vueltas.insert(0, ...)` agrega la vuelta al inicio de la lista, de modo que la más reciente aparezca arriba.

05 Desarrollo · Parte B — Formateo e interfaz

4 Formatear el tiempo

1. Convierte las centésimas a un texto `mm:ss.cc`. `padLeft(2, '0')` rellena con ceros a la izquierda:

LIB/MAIN.DART · DART

```
String _formatear(int centesimas) {
  final min = centesimas ~/ 6000;
  final seg = (centesimas ~/ 100) % 60;
  final cen = centesimas % 100;
  final mm = min.toString().padLeft(2, '0');
  final ss = seg.toString().padLeft(2, '0');
  final cc = cen.toString().padLeft(2, '0');
  return mm + ':' + ss + ':' + cc;
}
```

DEFINICIÓN · El operador `~/`

`~/` es la **división entera** en Dart (descarta los decimales). `%` es el resto. Con ambos separamos minutos, segundos y centésimas.

5 El cuerpo: tiempo y botones

1. En el `build`, muestra el tiempo grande y una `Row` con los tres botones:

LIB/MAIN.DART · DART

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Cronometro')),
    body: Column(
      children: [
        const SizedBox(height: 32),
        Text(
          _formatear(_centesimas),
          style: const TextStyle(
            fontSize: 56,
            fontWeight: FontWeight.bold,
          ),
        ),
        const SizedBox(height: 24),
        Row(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            ElevatedButton(
              onPressed: _iniciarPausar,
              child: Text(_corriendo ? 'Pausar' : 'Iniciar'),
            ),
            const SizedBox(width: 12),
          ],
        ),
      ],
    ),
  );
}
```

```

        ElevatedButton(
          onPressed: _vuelta,
          child: const Text('Vuelta'),
        ),
        const SizedBox(width: 12),
        ElevatedButton(
          onPressed: _reiniciar,
          child: const Text('Reiniciar'),
        ),
      ],
    ),
    // ... lista de vueltas en el siguiente paso
  ],
),
);
}

```

6 La lista de vueltas

1. Debajo de los botones, dentro de la [Column](#), agrega la lista de vueltas envuelta en [Expanded](#):

DART · DENTRO DE LA COLUMN

```

const SizedBox(height: 16),
Expanded(
  child: ListView.builder(
    itemCount: _vueltas.length,
    itemBuilder: (context, index) {
      return ListTile(
        leading: CircleAvatar(
          child: Text((_vueltas.length - index)
            .toString()),
        ),
        title: Text('Vuelta: ' + _vueltas[index]),
      );
    },
  ),
)

```

2. Ejecuta: inicia el cronómetro, marca algunas vueltas, pausa y reinicia. Verifica que el tiempo corre y las vueltas se registran.

06 Resultado esperado

Una pantalla con el tiempo en grande, tres botones y el historial de vueltas debajo. Comportamiento esperado:

- El tiempo avanza en formato **mm:ss:cc** mientras corre.
- El botón principal alterna entre "Iniciar" y "Pausar" según el estado.
- Cada "Vuelta" agrega el tiempo actual al inicio de la lista (solo si está corriendo).

- "Reiniciar" pone el tiempo en cero, detiene el conteo y limpia las vueltas.
- Al salir de la pantalla, el temporizador se cancela sin errores.

07 Verificación de resultados y entregable

Marca cada punto como verificación de que completaste el ejemplo:

- ☐ Se importó `dart:async` y se usa `Timer.periodic`.
- ☐ El tiempo se actualiza cada 100 ms con `setState`.
- ☐ El botón principal alterna Iniciar/Pausar correctamente.
- ☐ "Vuelta" registra el tiempo y "Reiniciar" limpia todo.
- ☐ El tiempo se muestra formateado como `mm:ss:cc`.
- ☐ El `Timer` se cancela en `dispose`.

EJEMPLO · Qué entregar

El archivo `lib/main.dart` terminado y un documento (PDF o Word) con: (1) captura del cronómetro corriendo; (2) captura con varias vueltas registradas; y (3) **reto personal**: resalta la vuelta más rápida y la más lenta con un color distinto en su `ListTile`, o cambia la precisión a milésimas. Descríbelo en 3–5 líneas.

08 Cuestionario

1. ¿Qué hace `Timer.periodic` y por qué se guarda el `Timer` en una variable?
2. ¿Por qué es imprescindible cancelar el `Timer` en `dispose`?
3. ¿Para qué sirven los operadores `~/` y `%` al formatear el tiempo?
4. ¿Qué diferencia hay entre `insert(0, ...)` y `add(...)` al registrar vueltas?
5. Explica cómo el botón principal sabe si debe mostrar "Iniciar" o "Pausar".

09 Rúbrica de evaluación

Criterio	Puntaje
<code>Timer.periodic</code> y actualización del tiempo	30 %
Iniciar/Pausar/Reiniciar funcionando	25 %
Formateo <code>mm:ss:cc</code> y registro de vueltas	20 %
Cancelación del <code>Timer</code> en <code>dispose</code>	10 %
Cuestionario resuelto	15 %

10 Referencias

- Dart — `Timer` (API `dart:async`). [api.dart.dev · Timer](https://api.dart.dev/Timer)
- Flutter — `StatefulWidget` lifecycle (`dispose`). [api.flutter.dev · State.dispose](https://api.flutter.dev/State.dispose)

- Flutter — Work with long lists (ListView.builder). docs.flutter.dev/cookbook/lists/long-lists

Ejemplo integrador de las Guías 1–5. Contenido basado en la documentación oficial de Flutter/Dart (v3.44). Elaborado para INF 662 — UATF.